

RAPPORT DE RÉUNION – PROJET SMARTBUS

Objet : Définition des règles de gestion et de l'architecture cible

Date : 17 juin 2026

Participants : Équipe projet SMARTBUS

1. Contexte et objectif de la réunion

La présente réunion avait pour objectif de définir précisément les **règles de gestion** relatives à l'inscription des écoles/organisations, à la répartition des données entre la **SUPER APP** et les **applications écoles**, ainsi qu'à la mise en place d'une architecture **scalable** et **maintenable** dans le temps.

À l'issue des échanges, les points suivants ont été validés et feront office de spécifications techniques pour la phase de développement.

2. Règles de gestion validées

2.1. Point d'entrée unique : l'APP École

- L'inscription d'une nouvelle organisation (école, garderie, centre de loisirs, etc.) se fait **exclusivement** via l'**application École** (interface web accessible au public).
- À ce stade, l'utilisateur n'a pas encore de base de données dédiée.

2.2. Remontée automatique vers la SUPER APP

- Dès qu'un utilisateur soumet son formulaire d'inscription (nom, email, mot de passe, informations de l'école) via l'APP École, les données sont **immédiatement transmises** à la **SUPER APP** via l'API centralisée.
- La SUPER APP enregistre alors ce nouvel utilisateur dans sa **base de données centrale** en tant que **nouveau client**.
- La base de données de la SUPER APP contient donc **l'intégralité des utilisateurs** inscrits sur la plateforme SMARTBUS.



Sud Contractors,
Abidjan Cocody Riviera3,
Boulevard Koffi Gadeau,
Immeuble Aziza 2, 4e étage

27 21 20 17 23
07 59 26 22 22



contact@sudcontractors.com



sudcontractors.com

• 3^e Prix National d'Excellence 2024, en Mobilité Urbaine et Sécurité Routière

• Vainqueur du Hackathon SIREXE 2024



PROVISION



PROVISION+



PROCAM



PROFUEL



PROTEMP

2.3. Création de l'organisation et génération automatique de la base de données dédiée

- Après validation de son inscription (et souscription d'un forfait), l'utilisateur peut finaliser la création de son organisation (définir le nom, l'adresse, les paramètres, etc.).
- À cet instant précis, **un script d'orchestration** est automatiquement exécuté par le système central. Ce script a pour mission de :
 1. **Générer une nouvelle base de données** (ou un nouveau schéma isolé) **dédiée à cette école**.
 2. **Exécuter le script de migration** (structure des tables) sur cette nouvelle base.
 3. **Initialiser les tables opérationnelles** qui permettront à l'école de gérer ses propres entités métier : ENFANT, PARENT, CHAUFFEUR, CAR, TRAJET, COURSE, etc.

2.4. Périmètres des bases de données

Base de données	Contenu	Gestion
SUPER APP (Centrale)	• Utilisateurs (toutes les écoles) • Organisations/Écoles • Forfaits souscrits • Rôles des utilisateurs • Historique des paiements/suspensions	Gérée par la SUPER APP. Ne contient pas de données opérationnelles (enfants, courses).
BASE ÉCOLE (Dédiée)	• Enfants • Parents • Chauffeurs • Cars • Trajets / Points d'arrêt • Courses • Événements biométriques	Gérée par l'école via son application. Une base par école (ou par groupe).

3. Architecture scalabilité & gestion des versions

Afin d'assurer une **grande scalabilité** et de permettre des mises à jour sans interruption de service, les règles suivantes ont été établies concernant la gestion des versions.

3.1. Problématique identifiée

- L'application École (frontend) et la base de données d'une école (schéma) peuvent évoluer indépendamment.
- Il est impératif de **vérifier la compatibilité** entre la version de l'application installée et la version du schéma de la base de données avant toute opération.

3.2. Solution retenue : Table meta_data (versionnage)

Nous mettrons en place une **table de métadonnées** (ou schema_version) dans **chaque base de données** (SUPER APP et bases écoles).

Structure de la table meta_data :

Champ	Type	Description
key	VARCHAR(100)	Clé unique (ex: 'schema_version', 'app_min_version')
value	VARCHAR(50)	Valeur correspondante (ex: '1.2.0')
updated_at	TIMESTAMP	Date de dernière mise à jour

Exemple d'enregistrements :

key	value
schema_version	2.1.0
app_min_version	2.0.5
last_migration	2026-06-17 10:30:00

3.3. Fonctionnement du mécanisme de mise à jour

1. **Initialisation** : Lors de la génération automatique de la base de données de l'école, la table meta_data est créée et renseignée avec la version actuelle du schéma.
2. **Connexion de l'application** : À chaque démarrage de l'APP École, celle-ci interroge la base de données pour récupérer schema_version.
3. **Vérification de compatibilité** :
 - Si la version de l'APP École est **inférieure** à la version requise par la base (app_min_version) → l'application bloque et demande une mise à jour.
 - Si la version de la base (schema_version) est **inférieure** à la version attendue par l'APP École → une procédure de mise à jour du schéma (migration) est déclenchée.
4. **Mise à jour progressive** :
 - Les migrations de schéma sont versionnées.
 - Un script de migration est exécuté automatiquement (ou semi-automatiquement) pour passer d'une version n à n+1.
 - La valeur de schema_version dans meta_data est incrémentée après succès de la migration.

3.4. Avantages de cette approche

Avantage	Description
Scalabilité	Chaque école évolue à son rythme. Toutes les bases ne sont pas mises à jour en même temps, ce qui réduit les risques.
Disponibilité	Les mises à jour peuvent être effectuées sans interruption de service (rolling updates).
Traçabilité	On sait exactement quelle version tourne sur quelle base.
Automatisation	Le script de génération de base inclut automatiquement la table meta_data.

4. Récapitulatif des responsabilités

Acteur / Module	Responsabilité
Utilisateur (APP École)	S'inscrit, crée son organisation, gère ses données opérationnelles.
SUPER APP	Centralise les utilisateurs, les organisations, les forfaits. Orchestre la création des bases écoles.
Script de provisionnement	Génère automatiquement la base de données de l'école et exécute les migrations initiales.
Table meta_data	Assure le versionnage du schéma et la compatibilité avec l'application.
API Gateway	Vérifie les accès (JWT) et redirige les requêtes vers la bonne base de données.

5. Stratégie de conception et de développement

5.1. Décision stratégique

La conception et le développement du projet démarreront en priorité sur la SUPER APP.

Cette décision a été validée à l'unanimité pour les raisons suivantes :

1. **La SUPER APP est le cœur du système**
C'est elle qui gère l'ensemble des utilisateurs, des organisations et des abonnements. Elle est le point d'entrée de toute la plateforme.
2. **La SUPER APP est l'orchestrateur**
C'est elle qui déclenche la création automatique des bases de données des écoles via le script de provisionnement. Sans SUPER APP, aucune école ne peut exister.
3. **La SUPER APP définit le socle technique**
Les API d'authentification, de gestion des utilisateurs, de vérification des accès (rôles, forfaits, suspensions) sont toutes portées par la SUPER APP. L'APP École et les autres composants en dépendent.
4. **La table meta_data et la gestion des versions sont d'abord implémentées dans la SUPER APP**
Le mécanisme de versionnage doit être opérationnel au niveau central avant d'être déployé sur les bases écoles.

5.2. Conséquences pour l'APP École

- L'APP École sera développée **après** la SUPER APP, car elle consomme les API de la SUPER APP pour :
 - L'authentification des utilisateurs.
 - L'inscription et la création des organisations.
 - La vérification du statut (actif / suspendu) avant toute opération.

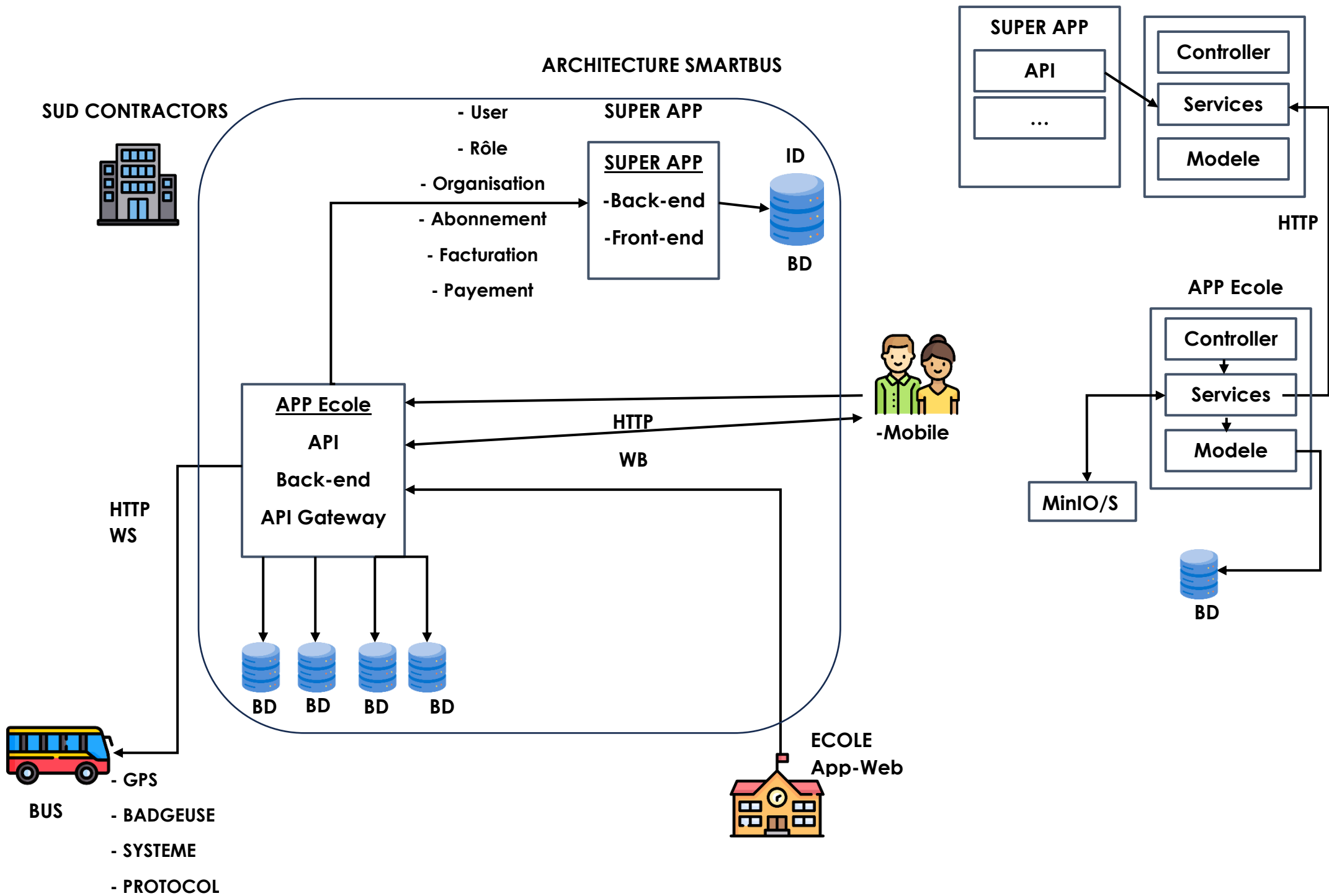
5.3. Ordre de développement recommandé

Ordre	Composant	Justification
1	SUPER APP (Backend + Base de données)	Cœur du système, orchestre tout le reste
2	Script de provisionnement	Dépend de la SUPER APP, génère les bases écoles
3	API Gateway & Authentification (JWT)	Centralise la sécurité avant de l'étendre
4	APP École	Consomme les API de la SUPER APP, s'appuie sur l'orchestration
5	APP Parent & APP Conducteur	Utilisateurs finaux, dépendent des données des écoles

6. Conclusion et prochaines étapes

Les règles de gestion validées aujourd'hui permettent de poser une architecture **robuste, isolée et scalable** :

- **Isolation totale** des données écoles (une base par client).
- **Supervision centralisée** via la SUPER APP (utilisateurs, forfaits, statuts).
- **Provisionnement automatique** (zéro intervention manuelle).
- **Gestion des versions** par métadonnées (table meta_data).
- **Priorité de développement** donnée à la SUPER APP pour garantir un socle solide.



COMMENTAIRE DE L'ARCHITECTURE SMARTBUS

1. PRINCIPE GÉNÉRAL

SMARTBUS repose sur une **architecture multi-tenant supervisée** avec deux niveaux distincts :

Niveau	Composant	Rôle
Central	SUPER APP	Supervision et gestion des clients (écoles)
Opérationnel	APP École	Gestion quotidienne de chaque établissement

2. GESTION DES UTILISATEURS ET DES ORGANISATIONS

Règle	Explication
Inscription unique	L'utilisateur s'inscrit sur l'APP École. Les données remontent automatiquement vers la SUPER APP.
Base centrale	La SUPER DB enregistre : les utilisateurs, les organisations, les forfaits et les rôles.
Création d'organisation	Après inscription, l'utilisateur crée son école et souscrit un forfait.

3. PROVISIONNEMENT AUTOMATIQUE DES BASES

Règle	Explication
Génération automatique	Dès qu'une organisation est créée, un script génère une base de données dédiée pour cette école.
Isolation totale	Chaque école a sa propre base (Enfant, Parent, Chauffeur, Car, Trajet, Course, Événement).
Aucun échange entre bases	Les bases écoles ne communiquent pas entre elles. L'API Gateway route les requêtes vers la bonne base selon l'organisation authentifiée.

4. GESTION DES VERSIONS (SCALABILITÉ)

Règle	Explication
Table MetaData	Chaque base (SUPER DB et bases écoles) contient une table meta_data pour stocker : schema_version, app_min_version, last_migration.
Vérification automatique	À chaque connexion, l'APP École compare sa version avec celle de la base. Si incompatible, une mise à jour est déclenchée.
Mise à jour progressive	Les bases écoles peuvent être mises à jour indépendamment les unes des autres (scalabilité).